The Portsmith Papers: A Hands-On Tour of PostgreSQL Beyond the Relational Model

The Portsmith Papers
- A Hands-On Tour of PostgreSQL Beyond the Relational Model
  - About This Book
  - Author

Chapter 1 — JSONB: Semi-Structured Data Without a Schema Tax
- Background
- The Scenario
- Exercise Goals
- Installation
  - 1 — PostgreSQL
  - 2 — Python 3.12 and psycopg
- Loading the Data
  - Create the database
  - Run the seed script
  - Verify the load
- Exercises
  - Exercise 1 — The Three Extraction Operators
  - Exercise 2 — Filtering with Containment and Key Existence
  - Exercise 3 — Missing Keys vs. NULL Values
  - Exercise 4 — GIN Indexes
  - Exercise 5 — Updating Documents in Place
  - Exercise 6 — Expanding Arrays with `jsonb_array_elements`
  - Exercise 7 — Path Queries with `jsonb_path_query`
- Summary — What You Should Now Know

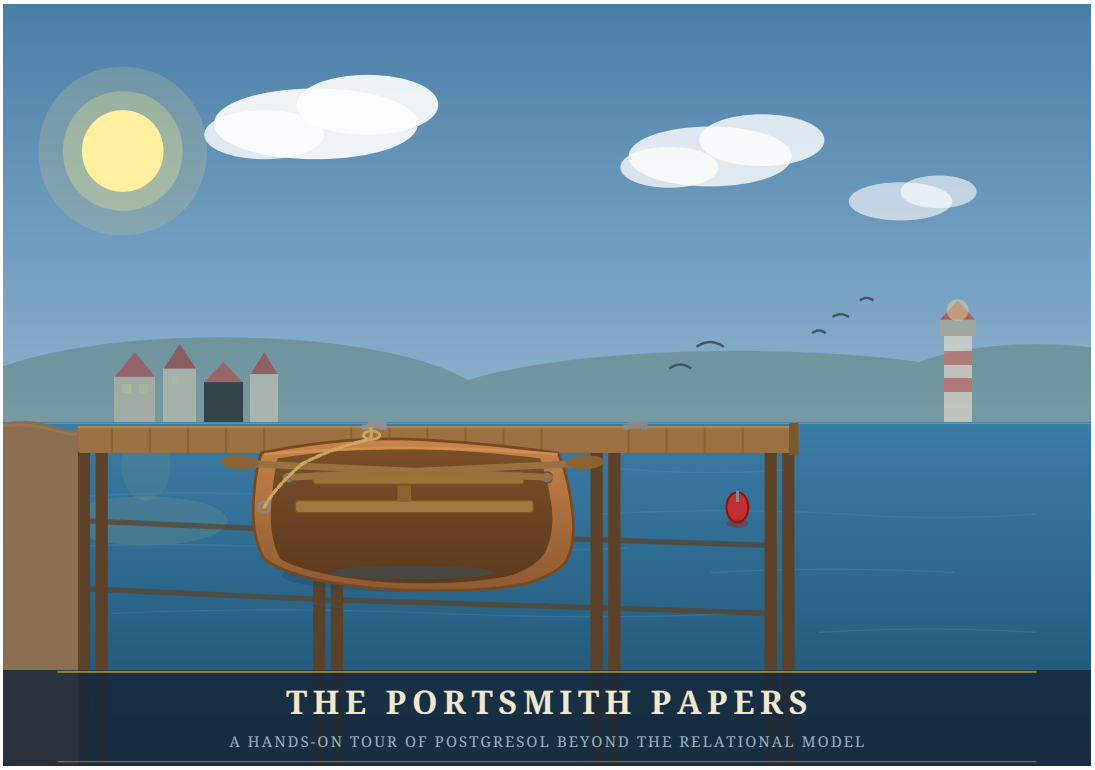
Chapter 2 — PostGIS: Geospatial Queries on Real Geometry
- Background
- The Scenario
- Exercise Goals
- Installation
  - 1 — PostGIS server package
  - 2 — Enable PostGIS in the database

- Loading the Data
  - Prerequisites
  - Run the Chapter 2 seed
  - Verify the load
- Exercises
  - Exercise 1 — Neighbourhood Polygons and WKT
  - Exercise 2 — Proximity Search with `ST_DWithin`
  - Exercise 3 — Spatial Joins with `ST_Within` and `ST_Contains`
  - Exercise 4 — Computing Area in Square Kilometres
  - Exercise 5 — Nearest Park Using `ST_Distance` and a Lateral Join
  - Exercise 6 — GIST Indexes and Spatial Query Plans
- Summary — What You Should Now Know

Chapter 3 — Job Queues: `FOR UPDATE SKIP LOCKED`
- Background
- The Scenario
- Exercise Goals
- Installation
- Loading the Data
  - Run the seed script
  - Verify the load
- Exercises
  - Exercise 1 — Designing the Queue Schema
  - Exercise 2 — The Atomic Claim Query
  - Exercise 3 — Simulating Concurrent Workers
  - Exercise 4 — Heartbeat and Stalled-Job Recovery
  - Exercise 5 — Dead-Lettering Exhausted Jobs
  - Exercise 6 — Benchmarking Claim Throughput with `pgbench`
- Summary — What You Should Now Know

The Portsmith Papers

A Hands-On Tour of PostgreSQL Beyond the Relational Model



> *"PostgreSQL is not a database with extensions.*
> *It is an extensible data platform that happens to speak SQL."*

About This Book

Most PostgreSQL tutorials end where the interesting work begins.

Once you know how to `SELECT`, `JOIN`, and `GROUP BY`, you have mastered perhaps twenty percent of what PostgreSQL can do. The remaining eighty percent — geospatial queries, semantic search, real-time notifications, fuzzy matching, vector embeddings, distributed coordination, and more — lives in a ecosystem of extensions, index types, and language features that most practitioners never discover.

This book is a guided tour of that other eighty percent.

Each chapter is built around a concrete engineering problem faced by the fictional city of **Portsmith** and its data platform team. We store business directories as semi-structured JSON documents. We route emergency services using geospatial proximity queries. We build a job queue with no message broker. We search municipal records with fuzzy matching that survives typos and OCR errors. We expose the entire platform as a REST API with zero application code.

Every chapter follows the same structure: synthetic data is generated first, giving you a realistic dataset to work against, and then a series of exercises walks you from first principles to production-ready technique. The exercises are written to be done — not just read.

By the end, you will see PostgreSQL not as a place to store rows, but as a programmable data infrastructure layer capable of doing work that most teams reach for separate specialized systems to handle.

What you will need:

- A working PostgreSQL 16 installation (setup covered in Appendix A)
- Python 3.12+
- A Debian-based Linux environment
- Familiarity with basic SQL (`SELECT`, `INSERT`, `JOIN`, `GROUP BY`)
- Curiosity about what else is in there

Author

Chris Lee

Edition 1.0 — Portsmith, 2026

Chapter 1 — JSONB: Semi-Structured Data Without a Schema Tax

> *"A schema is a prediction about the future. JSONB lets you hedge."*

Background

Relational databases enforce a contract: every row in a table has exactly the same columns. That contract is usually a feature — it catches mistakes and makes queries predictable. But sometimes you genuinely don't know what columns you'll need until the data arrives. A business directory is a good example. A restaurant has hours, cuisine, and a reservations policy. A hardware shop has trade accounts and parking. A hotel has star ratings and room types. Forcing them all into the same set of columns means drowning in NULLs or building a rats' nest of one-to-many extension tables.

PostgreSQL's `JSONB` type lets you store arbitrary JSON documents in a column while keeping the rest of your row fully relational. It is not a NoSQL escape hatch — it is an indexed, queryable, patchable document field that sits inside a SQL table. You can `WHERE` on it, join against it, aggregate over it, and update individual keys without rewriting the whole document.

This chapter works through all the operators and patterns you'll reach for most often. The exercises are short. Run every one, then try a variation of your own.

The Scenario

Portsmith's city council maintains a public business directory. Every business gets a short relational record — an ID, a name, an address, and a neighbourhood — but the supplementary detail varies so widely by business type that a single fixed schema would be unworkable.

The `businesses` table therefore stores all of that variable metadata in a `details` JSONB column. Each business category contributes its own keys:

Category	JSONB keys present
`restaurant`	`cuisine`, `price_range`, `rating`, `hours`, `tags`, `accepts_reservations`, `outdoor_seating`
`retail`	`subcategory`, `rating`, `hours`, `tags`, `payment_methods`, `has_parking`
`service`	`subcategory`, `rating`, `hours`, `tags`, `appointment_required`, `specialties`
`accommodation`	`subcategory`, `star_rating`, `rating`, `hours`, `amenities`, `room_types`, `price_per_night`
`entertainment`	`subcategory`, `rating`, `hours`, `tags`, `live_music`, `age_restriction`

The `hours` value is itself a nested object keyed by day of week (`mon`–`sun`). Each day is either `null` (closed) or `{"open": "HH:MM", "close": "HH:MM"}`. Several businesses also carry optional keys that appear only for their category, like `social` (social media handles), `happy_hour`, or `approved_brands`.

This heterogeneity is intentional — it is what makes the data a good vehicle for learning JSONB operators.

Exercise Goals

By the end of this chapter you will be able to:

- Extract values from a JSONB document using `->`, `->>`, and `#>>`, and explain why the operator choice changes the result type.
- Filter rows using the containment operator `@>` and the key-existence operators `?`, `?|`, and `?&`.
- Create a GIN index on a JSONB column and confirm that PostgreSQL uses it.
- Update a document in place with `jsonb_set` and the `||` merge operator without rewriting the whole column.
- Expand a JSONB array into a set of rows with `jsonb_array_elements` and aggregate the results.
- Write `jsonb_path_query` expressions to filter on nested values.

Installation

1 — PostgreSQL

If PostgreSQL 16 is not already installed:

```
sudo apt update
sudo apt install -y postgresql-16 postgresql-client-16
```

Confirm the server is running:

```
pg_lsclusters
```

You should see a cluster at version 16 in the `online` state.

2 — Python 3.12 and psycopg

```
sudo apt install -y python3.12 python3.12-venv
```

Create a virtual environment in the book's working directory and install the PostgreSQL driver:

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install "psycopg[binary]"
```

> **Note:** `psycopg` (version 3) is the modern Python driver for PostgreSQL. It is not available as an `apt` package for Python 3.12, so we install it via pip into the virtual environment. The `[binary]` extra bundles a pre-compiled C extension so you don't need `libpq-dev`.

Loading the Data

Create the database

```
# Create a role matching your OS user (skip if it already exists)
sudo -u postgres createuser --createdb "$(whoami)"

# Create the portsmith database
createdb portsmith
```

Run the seed script

From the `book/` directory, with the virtual environment active:

```
python data/ch01_seed.py
```

Expected output:

```
Connecting to: dbname=portsmith
Creating schema …
Inserting 48 businesses …
Done — 48 rows in businesses.
```

Verify the load

Open a `psql` session:

```
psql portsmith
```

Run these checks. If all three pass, the data is correct.

Check 1 — Row count and column types:

```
\d businesses
```

```
                                    Table "public.businesses"
    Column     |  Type   | Collation | Nullable |
Default
---------------+---------+-----------+----------
+-----------------------------------
 id            | integer |           | not null |
nextval('businesses_id_seq'::regclass)
 name          | text    |           | not null |
 address       | text    |           | not null |
 neighbourhood | text    |           | not null |
 details       | jsonb   |           | not null |
```

Check 2 — Counts by neighbourhood:

```
SELECT neighbourhood, COUNT(*) AS businesses
FROM   businesses
GROUP  BY neighbourhood
ORDER  BY neighbourhood;
```

```
   neighbourhood    | businesses
--------------------+------------
 Harbour District   |          9
 Industrial Port    |          7
 Northgate          |          9
 Old Town           |          9
 Riverside          |          9
```

```
 University Quarter |          5
(6 rows)
```

Check 3 — Counts by category:

```sql
SELECT details->>'category' AS category, COUNT(*) AS businesses
FROM   businesses
GROUP  BY category
ORDER  BY category;
```

```
    category     | businesses
-----------------+------------
 accommodation   |          5
 entertainment   |          6
 restaurant      |         15
 retail          |         12
 service         |         10
(5 rows)
```

If all three match, proceed to the exercises.

Exercises

Exercise 1 — The Three Extraction Operators

JSONB has two families of accessor. The arrow operators (`->` and `->>`) navigate one key at a time. The path operator (`#>>`) jumps straight to a deeply nested value. The difference between them is the **type they return**.

1.1 — Arrow operators

Run these two queries and compare the result types:

```sql
-- -> returns JSONB
SELECT name,
       details -> 'rating' AS rating_jsonb
FROM   businesses
LIMIT  5;

-- ->> returns TEXT
SELECT name,
       details ->> 'rating' AS rating_text
FROM   businesses
LIMIT  5;
```

Look carefully at the column headings in `psql` — one will show `jsonb`, the other `text`. This distinction matters the moment you try to do arithmetic:

```
-- This works: cast text to numeric
SELECT name,
       (details ->> 'rating')::numeric AS rating
FROM   businesses
ORDER  BY rating DESC
LIMIT  5;
```

```
          name           | rating
-------------------------+--------
 Lighthouse Bookshop     |    4.9
 Finch & Sons Barbers    |    4.9
 Portsmith Vet. Clinic   |    4.8
 River Bend Bakery       |    4.8
 Quarter Note Jazz Club  |    4.8
(5 rows)
```

> **Key point:** Use `->` when you want to keep working with JSONB (e.g., to navigate deeper). Use `->>` when you need the final value as text for comparisons, casting, or display.

1.2 — The path operator `#>>`

Navigating nested keys with chained `->` quickly becomes unreadable. The path operator takes an array of keys and returns the leaf as text in one step.

```
-- Chained arrows (verbose)
SELECT name,
       details -> 'hours' -> 'fri' ->> 'close' AS friday_close
FROM   businesses
WHERE  details ->> 'category' = 'restaurant'
LIMIT  8;

-- Path operator (equivalent, cleaner)
SELECT name,
       details #>> '{hours,fri,close}' AS friday_close
FROM   businesses
WHERE  details ->> 'category' = 'restaurant'
LIMIT  8;
```

Both produce the same result. Notice that some rows return `NULL` — those restaurants are closed on Fridays (the `fri` key is JSON `null`).

```
        name          | friday_close
----------------------+--------------
 The Gilded Clam      | 22:30
```

```
 Anchor & Oar Tavern | 01:00
 Bella Napoli        | 23:00
 Le Petit Bistro     | 14:30
 Dragon Palace       | 23:00
 Spice Garden        | 23:00
 Sol y Mar           | 23:00
 Mango Bay Caribbean | 22:30
(8 rows)
```

Exercise 2 — Filtering with Containment and Key Existence

2.1 — Containment: `@>`

The containment operator checks whether the left JSONB document contains all the key-value pairs in the right document. It is the most common way to filter on JSONB values.

Find all waterfront businesses:

```
SELECT name, neighbourhood
FROM   businesses
WHERE  details @> '{"tags": ["waterfront"]}'
ORDER  BY neighbourhood, name;
```

```
          name           |  neighbourhood
-------------------------+-----------------
 Anchor & Oar Tavern     | Harbour District
 Harbour Inn             | Harbour District
 Harbour View Theater    | Harbour District
 Mariners Rest B&B       | Harbour District
 Portsmith Fish Market   | Harbour District
 Saltbox Gallery         | Harbour District
 The Gilded Clam         | Harbour District
 Tidal Wave Surf Shop    | Harbour District
 Old Brewery Tap         | Industrial Port
 The Rusty Anchor        | Industrial Port
(10 rows)
```

> **Why this works:** The `@>` operator checks whether the tags *array* on the left contains the tags array `["waterfront"]` on the right — array containment is supported natively.

2.2 — Key existence: `?`

The `?` operator tests whether a key exists in a JSONB object (or a value exists in a JSONB array). Unlike `@>`, it does not check the value — only presence.

Find all businesses that publish social media handles:

```
        SELECT name,
               details -> 'social' AS social_links
        FROM   businesses
        WHERE  details ? 'social'
        ORDER  BY name;
```

```
          name          |
social_links
------------------------
+-----------------------------------------------------------
 Bella Napoli           | {"instagram": "@bellanapoli_portsmith"}
 Le Petit Bistro        | {"instagram": "@lepetitbistro"}
 Quarter Note Jazz Club | {"instagram": "@quarternote_portsmith"}
 Spice Garden           | {"instagram": "@spicegarden_portsmith"}
 The Gilded Clam        | {"facebook": "thegildedclam",
"instagram": "@gildedclam"}
 The Riverside Vegan    | {"instagram": "@riverside_vegan"}
(6 rows)
```

2.3 — Any-key and all-key existence: `?|` and `?&`

`?|` returns true if *at least one* of the listed keys exists. `?&` returns true only if *all* listed keys exist.

```
        -- Businesses that offer either delivery OR takeaway
        SELECT name
        FROM   businesses
        WHERE  details ?| ARRAY['delivery', 'takeaway']
        ORDER  BY name;

        -- Businesses that offer BOTH delivery AND takeaway
        SELECT name
        FROM   businesses
        WHERE  details ?& ARRAY['delivery', 'takeaway']
        ORDER  BY name;
```

Run both and note the difference in result count. The `?&` set is a strict subset of the `?|` set.

Exercise 3 — Missing Keys vs. NULL Values

JSONB has two distinct ways to represent "no value": a JSON `null` and a **missing key**. They behave differently in queries.

In our data, a business closed on Monday can be represented two ways:

- `"mon": null` — the key exists, the value is JSON null
- the `mon` key is absent entirely

The seed data uses `"mon": null` for closures, so we can observe this.

3.1 — The difference matters for `?`

```
-- ? checks key EXISTENCE, not value
SELECT name
FROM   businesses
WHERE  (details -> 'hours') ? 'sun'   -- key exists (even if null)
ORDER  BY name;
```

Run it. You should get **43 rows** — every business that uses a day-keyed hours object. The five accommodation businesses are excluded because their hours look like `{"reception": "07:00-23:00"}` and have no `sun` key.

Now try what seems like a natural follow-up:

```
--   This does NOT do what you might expect
SELECT name
FROM   businesses
WHERE  (details -> 'hours' -> 'sun') IS NOT NULL
ORDER  BY name;
```

Run it. You still get **43 rows** — identical to the first query.

Why? This is one of JSONB's most important gotchas:

> **JSONB `null` is not SQL `NULL`.**

When the seed script stores `"sun": None` in Python, `json.dumps` produces `"sun": null` — a JSON null *value* under an existing key. When PostgreSQL evaluates `details -> 'hours' -> 'sun'` on such a row, it returns the JSONB value `null`. That is a real value — not the absence of a value — so `IS NOT NULL` evaluates to `TRUE`.

SQL `NULL` only appears when the key itself is **missing** from the object. That is what happens for the five accommodation businesses: `details -> 'hours'` returns `{"reception": "..."}`, and then `-> 'sun'` on that object finds no such key and returns SQL `NULL`, so `IS NOT NULL` is `FALSE`.

To correctly distinguish the three states, use `jsonb_typeof()`:

State	`details -> 'hours' -> 'sun'`	`jsonb_typeof(...)`	`IS NOT NULL`
Key missing (accommodation)	SQL `NULL`	SQL `NULL`	`FALSE`
Key present, closed (`null`)	JSONB `null`	`'null'`	**`TRUE` ← gotcha**
Key present, open (object)	JSONB object	`'object'`	`TRUE`

The correct query for "businesses actually open on Sunday":

```
SELECT name
FROM   businesses
WHERE  jsonb_typeof(details -> 'hours' -> 'sun') = 'object'
ORDER  BY name;
```

This returns **25 rows** — only the businesses with a real hours object for Sunday. `jsonb_typeof` returns `'object'` only for JSON objects, `'null'` for JSON null, and SQL `NULL` for a missing key (which then fails the `=` test).

> **Rule of thumb:** Never use `IS NOT NULL` to test whether a JSONB navigation result is a "real" value. Use `jsonb_typeof(...)` to check the actual JSON type, or navigate one level deeper (e.g., check `details #>> '{hours,sun,open}'`) — the `#>>` operator converts JSON null to SQL NULL, so a deeper path naturally filters it out.

3.2 — Handling accommodation differently

Hotels and hostels store hours as a single `"reception": "HH:MM-HH:MM"` string rather than a day-keyed object. This means the `? 'sun'` test returns false for them even though they may be open 24/7. Write a query that finds all businesses that are either:

- Open on Sunday (day-keyed hours with a non-null `sun`), **or**
- An accommodation with 24/7 reception

```
SELECT name, neighbourhood,
       CASE
           WHEN jsonb_typeof(details -> 'hours' -> 'sun') =
'object'
               THEN (details #>> '{hours,sun,open}') || '–' ||
                    (details #>> '{hours,sun,close}')
           WHEN details #>> '{hours,reception}' = '24/7'
               THEN '24/7'
           ELSE details #>> '{hours,reception}'
```

```
        END AS sunday_availability
FROM   businesses
WHERE  jsonb_typeof(details -> 'hours' -> 'sun') = 'object'
   OR  (details -> 'hours') ? 'reception'
ORDER  BY neighbourhood, name;
```

Exercise 4 — GIN Indexes

Without an index, every JSONB query is a sequential scan — PostgreSQL reads every row and evaluates the expression. For a 48-row toy dataset that is unnoticeable, but with millions of rows it becomes a bottleneck.

A **GIN** (Generalised Inverted Index) index over a JSONB column indexes every key and value inside every document. It accelerates `@>`, `?`, `?|`, and `?&` queries.

4.1 — Observe the plan before indexing

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT name
FROM   businesses
WHERE  details @> '{"tags": ["waterfront"]}';
```

With 48 rows you will see `Seq Scan` (sequential scan, i.e. reading every row) — that is expected. On a larger table this would say `Seq Scan` with a high cost estimate.

4.2 — Create the GIN index

```
CREATE INDEX idx_businesses_details_gin
    ON businesses
    USING GIN (details);
```

4.3 — Observe the plan after indexing

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT name
FROM   businesses
WHERE  details @> '{"tags": ["waterfront"]}';
```

On a small table PostgreSQL may still choose a sequential scan (the planner knows the table fits in memory). Force it to use the index to see the mechanism:

```
SET enable_seqscan = off;

EXPLAIN (ANALYZE, BUFFERS)
SELECT name
```

```
FROM   businesses
WHERE  details @> '{"tags": ["waterfront"]}';

SET enable_seqscan = on;   -- always restore this
```

You should now see `Bitmap Index Scan on idx_businesses_details_gin` in the plan. The GIN index will pay for itself in production as the table grows.

> **What GIN indexes:** The default `jsonb_ops` operator class indexes every key and value in the document, supporting `@>`, `?`, `?|`, `?&`. A `jsonb_path_ops` class indexes only values (not keys), producing a smaller index that is faster for `@>` but cannot accelerate `?` queries. Use `jsonb_path_ops` when you only ever query by value containment and index size matters.

```
-- Alternative: value-only index (smaller, faster @>, no ? support)
CREATE INDEX idx_businesses_details_pathops
    ON businesses
    USING GIN (details jsonb_path_ops);
```

Exercise 5 — Updating Documents in Place

JSONB columns are immutable at the document level — you cannot change a single key without rewriting the whole value. PostgreSQL gives you two tools to do that rewrite concisely.

5.1 — `jsonb_set`: replace or add one value

The Lighthouse Bookshop just received a flood of five-star reviews. Update its rating to 5.0:

```
UPDATE businesses
SET    details = jsonb_set(details, '{rating}', '5.0')
WHERE  name = 'Lighthouse Bookshop';
```

Verify:

```
SELECT name, details ->> 'rating' AS rating
FROM   businesses
WHERE  name = 'Lighthouse Bookshop';
```

`jsonb_set(target, path, new_value)` — the path is a text array of keys. If the key already exists it is replaced; if it does not exist it is added (by default — there is a fourth parameter to suppress creation).

5.2 — `jsonb_set` on a nested key

The Gilded Clam is now closing an hour later on Sundays:

```
        UPDATE businesses
        SET    details = jsonb_set(
                             details,
                             '{hours,sun,close}',
                             '"21:00"'     -- note: a JSON string must be
double-quoted
                         )
        WHERE  name = 'The Gilded Clam';

        SELECT name,
               details #>> '{hours,sun,close}' AS new_sunday_close
        FROM   businesses
        WHERE  name = 'The Gilded Clam';
```

5.3 — `||` merge operator: add or overwrite multiple keys at once

The `||` operator merges two JSONB objects, with the right side winning on conflicts. Use it to add a `verified` flag and update `review_count` in a single statement:

```
        UPDATE businesses
        SET    details = details || '{"verified": true, "review_count":
999}'
        WHERE  name = 'Anchor & Oar Tavern';

        SELECT name,
               details ->> 'verified'     AS verified,
               details ->> 'review_count' AS reviews
        FROM   businesses
        WHERE  name = 'Anchor & Oar Tavern';
```

> **Caution:** `||` does a *shallow* merge. If `details` contains a nested object like `hours` and you merge another object that also has `hours`, the entire `hours` value is replaced — not deep-merged. Use `jsonb_set` for targeted nested updates.

Exercise 6 — Expanding Arrays with `jsonb_array_elements`

The `tags` key in most business documents is a JSON array. To query across tags — for example, to find the most common ones across the whole directory — you need to expand the array into individual rows.

6.1 — Expand one business's tags

```
        SELECT name,
               jsonb_array_elements_text(details -> 'tags') AS tag
```

```
        FROM   businesses
        WHERE  name = 'The Gilded Clam';
```

```
       name       |      tag
------------------+---------------
 The Gilded Clam  | waterfront
 The Gilded Clam  | seafood
 The Gilded Clam  | romantic
 The Gilded Clam  | outdoor_seating
(4 rows)
```

> `jsonb_array_elements_text` is a set-returning function: each element of the array becomes its own row. The result has more rows than the input.

6.2 — Count tags across all businesses

```
        SELECT tag, COUNT(*) AS occurrences
        FROM   businesses,
               jsonb_array_elements_text(details -> 'tags') AS tag
        GROUP  BY tag
        ORDER  BY occurrences DESC
        LIMIT  15;
```

```
       tag        | occurrences
------------------+-------------
 waterfront       |          10
 takeaway         |           5
 family_friendly  |           4
 family_owned     |           4
 organic          |           3
 ...
(15 rows)
```

Your exact numbers will differ — this shows the query pattern. The comma between `businesses` and the `jsonb_array_elements_text(...)` call is a **lateral join** (PostgreSQL expands the function for each input row automatically when used in the `FROM` clause this way).

6.3 — Only businesses with multiple specific tags

Find restaurants that are both `vegan_options` and `takeaway`:

```
        SELECT name, details ->> 'cuisine' AS cuisine
        FROM   businesses
        WHERE  details @> '{"tags": ["vegan_options"]}'
          AND  details @> '{"tags": ["takeaway"]}';
```

Because each `@>` call is independent, both conditions must hold. This is more efficient than expanding the array when the GIN index is in place.

Exercise 7 — Path Queries with `jsonb_path_query`

The `@?` operator and `jsonb_path_query()` function implement **SQL/JSON path language** — a mini-query language for navigating and filtering within a JSONB document. It is more expressive than chained arrow operators for conditional navigation.

7.1 — Simple path existence

Find businesses where the social links include Instagram:

```
        SELECT name
        FROM   businesses
        WHERE  details @? '$.social.instagram';
```

`$.social.instagram` means: start at the root (`$`), descend to `social`, then to `instagram`. `@?` returns true if the path resolves to at least one value.

7.2 — Filter on a nested value

Find restaurants with a rating above 4.5:

```
        SELECT name,
               details ->> 'rating' AS rating
        FROM   businesses
        WHERE  details @? '$ ? (@.category == "restaurant" && @.rating >
4.5)';
```

```
         name          | rating
-----------------------+--------
 Bella Napoli          | 4.6
 Le Petit Bistro       | 4.7
 River Bend Bakery     | 4.8
 Spice Garden          | 4.6
 The Gilded Clam       | 4.5
(5 rows)
```

The `? (filter)` syntax inside a path expression is equivalent to a `WHERE` clause inside the document.

7.3 — Find businesses open late on Friday

Businesses that close at or after 21:00 on Friday (using string comparison, which works correctly for 24-hour `HH:MM` strings):

```
        SELECT name,
               details #>> '{hours,fri,close}' AS friday_close
        FROM   businesses
```

```
        WHERE  details @? '$.hours.fri.close ? (@ >= "21:00")'
        ORDER  BY friday_close DESC, name;
```

```
           name           | friday_close
--------------------------+--------------
 Bella Napoli             | 23:00
 Dragon Palace            | 23:00
 Harbour View Theater     | 23:00
 The Hungry Scholar       | 23:00
 Spice Garden             | 23:00
 Sol y Mar                | 23:00
 Mango Bay Caribbean      | 22:30
 The Gilded Clam          | 22:30
 Thai Orchid              | 22:00
 The Riverside Vegan      | 22:00
 Northgate Grocers        | 21:00
 Lotus Spa & Wellness     | 20:00
(12 rows)
```

> **Limitation to notice:** Businesses that close *after midnight* — Anchor & Oar, The Clocktower Pub, The Rusty Anchor, and others — show a `close` of `"01:00"` or `"02:00"`. Lexicographically, `"02:00" < "21:00"`, so these businesses are *excluded* from the above results even though they are open very late. This is a design trade-off of storing times as plain strings. One solution is to store closing times past midnight as `"25:00"`, `"26:00"`, etc. — an unusual but practical convention for 24-hour time arithmetic.

7.4 — Extract values with `jsonb_path_query`

`@?` is a boolean test. `jsonb_path_query()` returns the actual matched values:

```
        SELECT name,
               jsonb_path_query(details, '$.hours.fri.close') AS
friday_close_json
        FROM   businesses
        WHERE  details @? '$.hours.fri'
        ORDER  BY name
        LIMIT  8;
```

The returned values are JSONB (note the quotes around the time strings). Use `jsonb_path_query_first(...) #>> '{}'` to extract a single scalar as text without the surrounding quotes.

Summary — What You Should Now Know

You have worked through the core JSONB toolkit. Here is what each operator and function you used actually does:

Tool	What it does
`-> 'key'`	Navigate one level; returns **JSONB**
`->> 'key'`	Navigate one level; returns **text**
`#>> '{a,b,c}'`	Navigate a path; returns **text**; JSON null becomes SQL NULL
`@> '{"k":"v"}'`	Containment test; accelerated by GIN
`? 'key'`	Key exists test (regardless of value); accelerated by GIN
`?|` / `?&`	Any-key / all-keys exist; accelerated by GIN
`jsonb_typeof(val)`	Returns `'object'`, `'array'`, `'string'`, `'number'`, `'boolean'`, or `'null'`; SQL NULL for a missing key
`jsonb_set(col, path, val)`	Replace or insert a nested value
`col || '{"k":"v"}'`	Shallow-merge a document
`jsonb_array_elements_text(col)`	Expand a JSON array into rows
`@? 'path'`	SQL/JSON path existence test
`jsonb_path_query(col, 'path')`	SQL/JSON path — return matched values

> **Remember:** JSONB `null` ≠ SQL `NULL`. Use `jsonb_typeof()` — not `IS NOT NULL` — when you need to distinguish a missing key from a key that exists but holds a JSON null value.

The key design insight from this chapter is that JSONB lets you keep a relational spine — the columns your queries always need (`id`, `name`, `neighbourhood`) — while storing everything else in a document whose shape can vary row by row. The GIN index means you do not pay a query performance penalty for that flexibility.

In the next chapter you will add point geometry to the `businesses` table and use PostGIS to answer spatial questions: which businesses are within 500 metres of the harbour, and which neighbourhood does each one belong to.

Going further: PostgreSQL 14+ supports the `jsonpath` type natively. The `jsonb_path_query_array` and `jsonb_path_query_first` variants are useful for pagination and single-value extraction. For write-heavy workloads,

profile whether `JSONB` or a computed stored column (Chapter 16) gives better `INSERT`/`UPDATE` throughput on your hardware.

Chapter 2 — PostGIS: Geospatial Queries on Real Geometry

> *“A city is not a list of rows. It is a shape on the ground.”*

Background

Every interesting question about a city is ultimately a spatial question. Which businesses are near the harbour? Which neighbourhood is this address in? How large is the industrial waterfront? Relational databases answer these badly when location is stored as a text field or a pair of float columns — there is no native concept of “within”, “contains”, or “distance”.

PostGIS is a PostgreSQL extension that adds first-class geometry and geography types, plus several hundred spatial functions and operators. It turns PostgreSQL into a full spatial database: you can store points, lines, and polygons; index them with GIST; and answer proximity, containment, and area queries in SQL without an external GIS system.

This matters in practice far beyond mapping applications. Address geocoding, logistics routing, fraud detection (is this login coming from the expected region?), real estate valuation, and urban planning all reach for spatial queries. PostGIS is the standard tool for all of them in the PostgreSQL ecosystem.

The Scenario

The Portsmith business directory from Chapter 1 stores each business’s neighbourhood as a plain text column. That works for simple filtering, but it cannot answer *where* questions: it cannot find businesses near a given coordinate, cannot verify that a business address actually falls inside its declared neighbourhood, and cannot measure distances.

This chapter adds a point geometry to every business record, then introduces three new spatial tables:

Table	Geometry type	What it holds
`neighborhoods`	`POLYGON`	Boundary polygons for Portsmith's six neighbourhoods
`parks`	`POLYGON`	Six public parks and green spaces
`city_infrastructure`	`LINESTRING`	Twelve named road segments

All coordinates are in WGS-84 (SRID 4326), the same coordinate system used by GPS and most web mapping APIs.

Exercise Goals

By the end of this chapter you will be able to:

- Store and inspect `POLYGON` geometry using WKT (Well-Known Text) and `ST_GeomFromText`.
- Run proximity searches with `ST_DWithin`, and understand why the `::geography` cast matters for distance accuracy.
- Perform spatial joins using `ST_Within` and `ST_Contains`, and explain the difference between them.
- Compute polygon areas in square kilometres using `ST_Area` with the geography type.
- Find the nearest feature for every row using `ST_Distance` and a `CROSS JOIN LATERAL`.
- Create a GIST spatial index and confirm that PostgreSQL uses it.

Installation

1 — PostGIS server package

PostGIS is a separate package from PostgreSQL. On Debian/Ubuntu:

```
sudo apt install -y postgresql-16-postgis-3
```

2 — Enable PostGIS in the database

Connect to the `portsmith` database and enable the extension:

```
psql portsmith
```

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

Verify it loaded:

```
SELECT postgis_full_version();
```

You should see a long string beginning with `POSTGIS="3.x.x"`. If you see an error about the extension not existing, the server package is not installed.

Loading the Data

Prerequisites

Chapter 1's seed script must have been run first — the `businesses` table must exist. If it does not:

```
python data/ch01_seed.py
```

Run the Chapter 2 seed

```
python data/ch02_seed.py
```

Expected output:

```
Connecting to: dbname=portsmith
Enabling PostGIS extension …
Applying DDL …
Inserting 6 neighbourhoods …
Inserting 6 parks …
Inserting 12 road segments …
Updating 48 business locations …

Done:
  businesses with geometry : 48
  neighbourhoods           : 6
  parks                    : 6
  road segments            : 12
```

Verify the load

Open `psql portsmith` and run these four checks.

Check 1 — businesses now has a geometry column:

```
\d businesses
```

You should see a new `geom` column of type `geometry(Point,4326)`.

Check 2 — neighbourhood table structure and row count:

```
        SELECT name, population,
               ST_AsText(geom) AS wkt_preview
        FROM   neighborhoods
        ORDER  BY name;
```

```
       name          | population |            wkt_preview
--------------------+------------
+-----------------------------------
 Harbour District    |       4200 | POLYGON((-1.805 50.69,...))
 Industrial Port     |       2100 | POLYGON((-1.77 50.69,...))
 Northgate           |      18500 | POLYGON((-1.83 50.732,...))
 Old Town            |       6800 | POLYGON((-1.805 50.71,...))
 Riverside           |       9300 | POLYGON((-1.773 50.71,...))
 University Quarter  |      11200 | POLYGON((-1.83 50.71,...))
(6 rows)
```

Check 3 — parks and roads:

```
        SELECT COUNT(*) FROM parks;
        SELECT COUNT(*) FROM city_infrastructure;
```

Both should return 6 and 12 respectively.

Check 4 — SRID on all tables:

```
        SELECT f_table_name, f_geometry_column, srid, type
        FROM   geometry_columns
        WHERE  f_table_schema = 'public'
        ORDER  BY f_table_name;
```

```
      f_table_name      | f_geometry_column | srid |    type
------------------------+-------------------+------+------------
 businesses             | geom              | 4326 | POINT
 city_infrastructure    | geom              | 4326 | LINESTRING
 neighborhoods          | geom              | 4326 | POLYGON
 parks                  | geom              | 4326 | POLYGON
(4 rows)
```

If all four pass, proceed to the exercises.

Exercises

Exercise 1 — Neighbourhood Polygons and WKT

Geometry in PostGIS is often loaded from **Well-Known Text** (WKT), a human-readable representation of shapes. Understanding WKT lets you read geometry from SQL output, write it in queries, and reason about what you stored.

1.1 — Read a polygon in WKT

```
        SELECT name, ST_AsText(geom) AS wkt
        FROM   neighborhoods
        WHERE  name = 'Harbour District';
```

```
       name        |                          wkt
------------------
+-----------------------------------------------------
 Harbour District  | POLYGON((-1.805 50.69,-1.77 50.69,-1.77
50.71,...))
```

WKT for a polygon is `POLYGON((x1 y1, x2 y2, ...))`. In geographic coordinates the convention is *longitude latitude* (x then y), matching the X/Y convention in maths. The ring must **close** — the last coordinate must equal the first.

1.2 — Inspect the bounding box

`ST_Envelope` returns the minimum bounding rectangle for any geometry:

```
        SELECT name,
               ST_XMin(ST_Envelope(geom)) AS west,
               ST_XMax(ST_Envelope(geom)) AS east,
               ST_YMin(ST_Envelope(geom)) AS south,
               ST_YMax(ST_Envelope(geom)) AS north
        FROM   neighborhoods
        ORDER  BY name;
```

Use this output to confirm each polygon sits in the right part of the coordinate space (longitudes around −1.75 to −1.83, latitudes around 50.69 to 50.76).

1.3 — Count vertices

```
        SELECT name,
               ST_NPoints(geom) AS vertex_count
        FROM   neighborhoods
        ORDER  BY name;
```

Each neighbourhood polygon has five vertices (four corners plus the closing duplicate). A real city boundary imported from an OS/census shapefile might have thousands.

1.4 — Understand the SRID

`ST_SRID` returns the spatial reference identifier stored with the geometry:

```
SELECT name, ST_SRID(geom) AS srid
FROM   neighborhoods
LIMIT  3;
```

SRID 4326 is the WGS-84 system used by GPS. Every geometry in these tables carries the same SRID, which means they can be compared and joined directly. If SRIDs differ, PostGIS will return an error — a deliberate safety check.

> **The geometry/geography distinction:** In PostGIS, `geometry` stores coordinates in whatever units the SRS defines. For SRID 4326 that means degrees. When you ask for a distance between two `geometry` points in 4326, you get a value in *degrees* — nearly useless for human-scale distances. The `geography` type, by contrast, always works on a spheroid and returns distances in *metres*. You can cast any SRID-4326 geometry to geography with `::geography` to get metre-based calculations. The exercises use this cast throughout.

Exercise 2 — Proximity Search with `ST_DWithin`

`ST_DWithin(a, b, distance)` returns true when the distance between `a` and `b` is at most `distance`. With geography inputs the distance is in metres.

2.1 — Find businesses within 500 m of Portsmith Pier

Portsmith's main pier entrance sits at approximately (−1.785, 50.700). Find every business within 500 metres of it:

```
SELECT b.name,
       b.neighbourhood,
       ROUND(
           ST_Distance(b.geom::geography,
                       ST_Point(-1.785,
50.700)::geography)::numeric
       ) AS distance_m
FROM   businesses b
```

```
        WHERE  ST_DWithin(b.geom::geography,
                          ST_Point(-1.785, 50.700)::geography,
                          500)
        ORDER  BY distance_m;
```

```
          name           |  neighbourhood   | distance_m
-------------------------+------------------+------------
 The Gilded Clam         | Harbour District |         70
 Anchor & Oar Tavern     | Harbour District |        179
 Tidal Wave Surf Shop    | Harbour District |        307
 Harbour Inn             | Harbour District |        437
 Portsmith Fish Market   | Harbour District |        484
(5 rows)
```

All five results are in the Harbour District — exactly what you would expect for a search centred on the pier.

2.2 — Why `::geography` matters

Try the same query without the cast. Note that `ST_Point(...)` without an explicit SRID gets SRID 0, which PostGIS refuses to compare against SRID-4326 geometry — so you must use `ST_SetSRID`:

```
        SELECT b.name,
               ROUND(ST_Distance(b.geom,
                                 ST_SetSRID(ST_Point(-1.785, 50.700),
4326))::numeric,
                     6)                AS distance_degrees
        FROM   businesses b
        WHERE  ST_DWithin(b.geom,
                          ST_SetSRID(ST_Point(-1.785, 50.700), 4326),
                          500)
        ORDER  BY distance_degrees;
```

The `WHERE` clause now has a threshold of `500` — but in *degrees*. One degree of latitude is about 111 km, so this matches businesses within roughly 55,000 km: all 48 of them, from the whole city. The `distance_degrees` values in the output are tiny fractions like `0.001000`, not useful numbers.

```
rows returned: 48  (the entire city, not 5)
```

> **Rule:** For any distance query on SRID-4326 data, always cast to `::geography` in `ST_DWithin` and `ST_Distance`. The cast has negligible performance cost at the scales used in city-level data.

2.3 — Try different radii

How many businesses fall within 1 km of the pier? 2 km?

```
        SELECT radius_m,
               COUNT(*) AS business_count
        FROM   (VALUES (500), (1000), (2000)) AS radii(radius_m)
        CROSS JOIN LATERAL (
            SELECT 1
            FROM   businesses b
            WHERE  ST_DWithin(b.geom::geography,
                              ST_Point(-1.785, 50.700)::geography,
                              radius_m)
        ) AS matches
        GROUP  BY radius_m
        ORDER  BY radius_m;
```

```
 radius_m | business_count
----------+----------------
      500 |              5
     1000 |              8
     2000 |             17
(3 rows)
```

The pier sits in the southern Harbour District. Doubling the radius to 2 km pulls in more of the central neighbourhoods, but the northern Northgate businesses are nearly 5 km away — a reminder that Portsmith stretches a significant distance north from the waterfront.

Exercise 3 — Spatial Joins with `ST_Within` and `ST_Contains`

A **spatial join** links rows from two tables based on a geometric relationship rather than a key match. The most common are containment tests: does point A fall inside polygon B?

3.1 — Which neighbourhood is each business in?

```
        SELECT b.name,
               b.neighbourhood          AS declared_neighbourhood,
               n.name                   AS postgis_neighbourhood
        FROM   businesses  b
        JOIN   neighborhoods n ON ST_Within(b.geom, n.geom)
        ORDER  BY n.name, b.name;
```

`ST_Within(a, b)` returns true when geometry `a` lies completely inside geometry `b`. The query should return all 48 businesses, each matched to its neighbourhood.

Confirm the declared `neighbourhood` column always matches `postgis_neighbourhood`:

```
        SELECT COUNT(*) AS mismatches
        FROM   businesses  b
```

```
        JOIN   neighborhoods n ON ST_Within(b.geom, n.geom)
        WHERE  b.neighbourhood <> n.name;

 mismatches
------------
          0
(1 row)
```

Zero mismatches: the text column from Chapter 1 and the geometry are consistent.

3.2 — `ST_Contains` is the inverse

`ST_Contains(a, b)` returns true when polygon `a` contains geometry `b` — it is the mirror of `ST_Within`. These two queries produce identical results:

```
        -- Point inside polygon
        SELECT b.name FROM businesses b
        JOIN   neighborhoods n ON ST_Within(b.geom, n.geom)
        WHERE  n.name = 'Old Town'
        ORDER  BY b.name;

        -- Polygon contains point
        SELECT b.name FROM businesses b
        JOIN   neighborhoods n ON ST_Contains(n.geom, b.geom)
        WHERE  n.name = 'Old Town'
        ORDER  BY b.name;

          name
--------------------------
 Bella Napoli
 Finch & Sons Barbers
 Le Petit Bistro
 Old Town Hardware
 Portsmith Accountancy Ltd.
 Portsmith Arms Hotel
 Portsmith Legal Group
 Portsmith Tailors
 The Clocktower Pub
(9 rows)
```

Both return the same 9 Old Town businesses.

> **The subtle difference:** `ST_Contains(A, B)` requires that no point of B lies on the boundary of A. A point sitting exactly *on* a polygon edge would fail `ST_Contains` but pass `ST_Covers`. In practice, coordinates rarely land precisely on a boundary, so the

> two functions behave identically for most point-in-polygon work. Reach for `ST_Covers` if you need to include boundary-touching cases explicitly.

3.3 — Check for gaps: businesses in no neighbourhood

This query finds any businesses whose geometry falls outside every neighbourhood polygon — useful for catching data quality problems:

```
        SELECT b.name, b.neighbourhood
        FROM   businesses b
        WHERE  NOT EXISTS (
            SELECT 1
            FROM   neighborhoods n
            WHERE  ST_Within(b.geom, n.geom)
        );
```

```
 name | neighbourhood
------+---------------
(0 rows)
```

All 48 businesses are inside a neighbourhood polygon.

Exercise 4 — Computing Area in Square Kilometres

`ST_Area` returns the area of a polygon. Called on `geometry` (degrees), it returns a value in square degrees — meaningless to most people. Called on `geography`, it returns square metres.

4.1 — Incorrect: area in square degrees

```
        SELECT name,
               ROUND(ST_Area(geom)::numeric, 6) AS area_sq_degrees
        FROM   neighborhoods
        ORDER  BY area_sq_degrees DESC;
```

The numbers look tiny (around `0.0007`). They are geometrically correct but impossible to interpret as real-world area because degrees are not uniform units.

4.2 — Correct: area in square metres, then kilometres

```
        SELECT name,
               ROUND((ST_Area(geom::geography) / 1e6)::numeric, 2) AS
area_km2
        FROM   neighborhoods
        ORDER  BY area_km2 DESC;
```

```
        name        | area_km2
--------------------+----------
 Northgate          |    17.59
 Old Town           |     5.53
 Harbour District   |     5.50
 University Quarter |     4.32
 Riverside          |     3.98
 Industrial Port    |     3.14
(6 rows)
```

Northgate is by far the largest neighbourhood — it spans the entire northern width of the city. Industrial Port is the smallest, a tight strip of dockside land. Old Town and Harbour District are almost identical in area despite having very different characters.

> **Why the numbers differ slightly from manual estimates:** `ST_Area` on geography uses the WGS-84 spheroid, which accounts for the fact that the Earth is not a perfect sphere. At latitude 50.7° the correction is small (under 0.3%) but present.

4.3 — Population density

Combine the computed area with the `population` column:

```
        SELECT name,
               population,
               ROUND((ST_Area(geom::geography) / 1e6)::numeric, 2) AS
area_km2,
               ROUND(
                   (population / (ST_Area(geom::geography) /
1e6))::numeric
               ) AS pop_per_km2
        FROM   neighborhoods
        ORDER  BY pop_per_km2 DESC;
```

```
        name        | population | area_km2 | pop_per_km2
--------------------+------------+----------+-------------
 University Quarter |      11200 |     4.32 |        2592
 Riverside          |       9300 |     3.98 |        2340
 Old Town           |       6800 |     5.53 |        1230
 Northgate          |      18500 |    17.59 |        1052
 Harbour District   |       4200 |     5.50 |         763
 Industrial Port    |       2100 |     3.14 |         668
(6 rows)
```

The University Quarter and Riverside are the most densely populated neighbourhoods — student housing and riverside apartments pack a lot of people into compact areas. The Industrial Port is the least dense despite its small size; much of it is warehouse and dock rather than residential.

Exercise 5 — Nearest Park Using `ST_Distance` and a Lateral Join

Finding the nearest feature from another table requires a **lateral join** — a subquery that can reference columns from the outer query row by row.

5.1 — Distance from one business to one park

`ST_Distance` between two geography values returns metres:

```
        SELECT b.name                              AS business,
               p.name                              AS park,
               ROUND(ST_Distance(b.geom::geography,
                                 p.geom::geography)::numeric) AS
distance_m
        FROM   businesses b,
               parks p
        WHERE  b.name = 'The Gilded Clam'
        ORDER  BY distance_m;
```

```
     business     |            park             | distance_m
------------------+-----------------------------+------------
 The Gilded Clam  | Harbourside Park            |        682
 The Gilded Clam  | Dockside Green              |       1315
 The Gilded Clam  | Market Square Gardens       |       2143
 The Gilded Clam  | Riverside Walk Park         |       2899
 The Gilded Clam  | University Grounds          |       3060
 The Gilded Clam  | Northgate Recreation Ground |       5006
(6 rows)
```

The nearest park to The Gilded Clam is Harbourside Park, 682 m away. The `ST_Distance` to a polygon returns the distance from the point to the nearest point on the polygon's boundary — zero when the point is inside the polygon.

5.2 — Nearest park for a single business (lateral pattern)

The canonical pattern for "nearest one thing" is `ORDER BY ... LIMIT 1` inside a lateral subquery:

```
        SELECT b.name,
               nearest.park_name,
               nearest.distance_m
        FROM   businesses b
        CROSS JOIN LATERAL (
            SELECT p.name                                          AS
park_name,
                   ROUND(ST_Distance(b.geom::geography,
```

```
                                        p.geom::geography)::numeric)      AS
distance_m
            FROM   parks p
            ORDER  BY b.geom::geography <-> p.geom::geography
            LIMIT  1
        ) AS nearest
        WHERE  b.name = 'Quarter Note Jazz Club';

         name          |    park_name       | distance_m
-----------------------+--------------------+------------
 Quarter Note Jazz Club | University Grounds |        233
(1 row)
```

The `<->` operator is the KNN (k-nearest-neighbour) distance operator. When used in an `ORDER BY` clause inside a lateral join, PostGIS can accelerate it with the GIST index (which you will create in Exercise 6). Using `<->` in `ORDER BY` is preferred over `ORDER BY ST_Distance(...)` for this reason.

5.3 — Nearest park for every business

Remove the `WHERE` filter to run across all 48 businesses:

```
        SELECT b.name                           AS business,
               b.neighbourhood,
               nearest.park_name,
               nearest.distance_m
        FROM   businesses b
        CROSS JOIN LATERAL (
            SELECT p.name                                          AS
park_name,
                   ROUND(ST_Distance(b.geom::geography,
                                        p.geom::geography)::numeric)      AS
distance_m
            FROM   parks p
            ORDER  BY b.geom::geography <-> p.geom::geography
            LIMIT  1
        ) AS nearest
        ORDER  BY b.neighbourhood, b.name;
```

Scan the results. You should see a clean pattern: each neighbourhood's businesses point to the park in that same neighbourhood. Notice that three Riverside businesses (Portsmith Pharmacy, Portsmith Veterinary Clinic, Riverside Cinema) and University Bookshop all show `distance_m = 0` — their coordinates fall *inside* the park polygon. `ST_Distance` to a polygon returns zero when the point is contained within it.

5.4 — Aggregate: average walking distance to a park, by neighbourhood

```
        SELECT b.neighbourhood,
               ROUND(AVG(nearest.distance_m)::numeric) AS avg_distance_m
        FROM   businesses b
        CROSS JOIN LATERAL (
            SELECT ROUND(ST_Distance(b.geom::geography,
                                     p.geom::geography)::numeric) AS
distance_m
            FROM   parks p
            ORDER  BY b.geom::geography <-> p.geom::geography
            LIMIT  1
        ) AS nearest
        GROUP  BY b.neighbourhood
        ORDER  BY avg_distance_m;
```

This gives a rough "walkability to green space" metric per neighbourhood. Old Town ranks first — Market Square Gardens sits centrally within the neighbourhood. Northgate comes last despite having the largest park, because the park is in the far north of the district while businesses cluster near the southern edge.

Exercise 6 — GIST Indexes and Spatial Query Plans

Without an index, every spatial query is a sequential scan: PostgreSQL reads every row, applies the geometry function, and discards non-matching rows. For a 48-row dataset that is instant, but at 48 million rows it is not.

PostGIS spatial queries are accelerated by **GIST** (Generalised Search Tree) indexes. A GIST index on a geometry column builds a tree of bounding boxes, allowing the planner to quickly eliminate large portions of the table.

6.1 — Observe the plan before indexing

```
        EXPLAIN (ANALYZE, BUFFERS)
        SELECT name
        FROM   businesses
        WHERE  ST_DWithin(geom::geography,
                          ST_Point(-1.785, 50.700)::geography,
                          500);
```

On the 48-row `businesses` table you will see something like:

```
Seq Scan on businesses  (cost=0.00..4.60 rows=1 width=...) ...
  Filter: (st_dwithin(...))
  Rows Removed by Filter: 43
```

A sequential scan is expected on a tiny table — the planner knows the overhead of an index lookup exceeds the cost of reading all 48 rows.

6.2 — Create GIST indexes on the geometry columns

```
CREATE INDEX idx_businesses_geom
    ON businesses USING GIST (geom);

CREATE INDEX idx_neighborhoods_geom
    ON neighborhoods USING GIST (geom);

CREATE INDEX idx_parks_geom
    ON parks USING GIST (geom);

CREATE INDEX idx_city_infrastructure_geom
    ON city_infrastructure USING GIST (geom);
```

6.3 — The geometry/geography index split

With `enable_seqscan` off, verify which plan the proximity query uses:

```
SET enable_seqscan = off;

EXPLAIN (ANALYZE, BUFFERS)
SELECT name
FROM   businesses
WHERE  ST_DWithin(geom::geography,
                  ST_Point(-1.785, 50.700)::geography,
                  500);

SET enable_seqscan = on;
```

You will see — perhaps surprisingly — that the planner *still* uses a sequential scan, even though `idx_businesses_geom` exists:

```
Seq Scan on businesses  (cost=10000000000.00...) ...
  Filter: st_dwithin((geom)::geography, ...)
```

Why? The GIST index was built on `geom` (type `geometry`). The query filters on `geom::geography` (type `geography`). These are different types — the index is not usable for geography operations.

To accelerate geography-based distance queries you need a **functional index** on the cast:

```
CREATE INDEX idx_businesses_geom_geography
    ON businesses USING GIST (CAST(geom AS geography));
```

Now with `enable_seqscan` off:

```
        SET enable_seqscan = off;

        EXPLAIN (ANALYZE, BUFFERS)
        SELECT name
        FROM   businesses
        WHERE  ST_DWithin(geom::geography,
                          ST_Point(-1.785, 50.700)::geography,
                          500);

        SET enable_seqscan = on;
```

```
Index Scan using idx_businesses_geom_geography on businesses
  Index Cond: ((geom)::geography &&
               _st_expand('...', '500'))
  Filter: st_dwithin((geom)::geography, ...)
  Rows Removed by Filter: 2
```

The index is now used. The index condition uses `&&` (bounding-box overlap on the spheroid) to quickly discard most rows, then `st_dwithin` rechecks the exact distance for the survivors.

6.4 — Verify the spatial join plan

The containment join uses geometry-to-geometry operations, so `idx_businesses_geom` (the plain geometry index) *is* used there:

```
        SET enable_seqscan = off;

        EXPLAIN (ANALYZE, BUFFERS)
        SELECT b.name, n.name AS neighbourhood
        FROM   businesses b
        JOIN   neighborhoods n ON ST_Within(b.geom, n.geom);

        SET enable_seqscan = on;
```

```
Nested Loop ...
  ->  Seq Scan on neighborhoods n  (6 rows — tiny table)
  ->  Index Scan using idx_businesses_geom on businesses b
        Index Cond: (geom @ n.geom)
        Filter: st_within(geom, n.geom)
```

For each neighbourhood polygon, `idx_businesses_geom` is used to find businesses whose bounding box falls inside the neighbourhood's bounding box (`geom @ n.geom`), then `st_within` rechecks the exact containment.

> **The index rule:** A GIST index on `geom geometry` accelerates geometry-to-geometry operations (`ST_Within`, `ST_Contains`, `ST_Intersects`, `&&`). A GIST index on `CAST(geom AS geography)` accelerates geography operations

> (`ST_DWithin(...::geography, ...)`, `<->` on geography). If you query both ways, create both indexes — they coexist happily on the same table.

Summary — What You Should Now Know

You have worked through the core PostGIS toolkit for points, polygons, and linear features. Here is a reference for everything used:

Function / operator	What it does
`ST_GeomFromText(wkt, srid)`	Parse WKT into a geometry with the given SRID
`ST_AsText(geom)`	Format geometry as WKT for display
`ST_SetSRID(ST_MakePoint(lon, lat), srid)`	Construct a point geometry
`ST_Point(lon, lat)::geography`	Construct a point and cast to geography
`geom::geography`	Cast a 4326 geometry to geography (distances now in metres)
`ST_SRID(geom)`	Return the SRID stored with a geometry
`ST_NPoints(geom)`	Count vertices in a geometry
`ST_Envelope(geom)`	Return the bounding-box rectangle
`ST_XMin/XMax/YMin/YMax(geom)`	Extract bounding-box extents
`ST_DWithin(a, b, d)`	True when the distance between a and b is ≤ d
`ST_Distance(a, b)`	Distance between two geometries (metres for geography)
`a::geography <-> b::geography`	KNN distance operator; use in ORDER BY for index acceleration
`ST_Within(a, b)`	

Function / operator	What it does
	True when a lies completely inside b
`ST_Contains(a, b)`	True when a contains b (inverse of ST_Within)
`ST_Area(geom::geography)`	Area in square metres on the spheroid
`CROSS JOIN LATERAL (... LIMIT 1)`	Nearest-neighbour join pattern
`USING GIST (geom)`	Create a GIST spatial index

> **Geometry vs geography in one sentence:** Use `geometry` for storage and when working with projected coordinate systems where units are already metres or feet. Cast to `geography` any time you need distance, area, or proximity results in real-world units from SRID-4326 data.

The coordinates added in this chapter will carry forward. Chapter 12 extends the `city_infrastructure` road network into a graph and uses a recursive CTE to find the shortest path between two intersections.

Going further: PostGIS supports many more geometry types — `MULTIPOLYGON` for areas with holes, `GEOMETRYCOLLECTION` for mixed types, and 3D geometries with a Z coordinate for elevation data. For routing specifically, `pgRouting` builds on PostGIS to provide Dijkstra, A, and turn-restriction-aware shortest-path algorithms over road network graphs. For importing real boundary data, `shp2pgsql` converts ESRI Shapefiles directly into PostGIS-compatible `INSERT` statements, and `ogr2ogr` handles GeoJSON, KML, GeoPackage, and dozens of other formats.*

Chapter 3 — Job Queues: `FOR UPDATE SKIP LOCKED`

> *“A queue is just a table that everyone is racing to read.”*

Background

Sooner or later almost every application needs a queue: a list of work items that a pool of workers processes one at a time, safely, without two workers ever grabbing the same item. The reflexive answer is to reach for a message broker — Redis, RabbitMQ, SQS, Kafka. Those tools earn their keep at serious scale. But if your data already lives in PostgreSQL, running a second system just to hand out rows to workers is often unnecessary complexity: another service to deploy, monitor, and keep consistent with the database.

PostgreSQL can do this job itself. The `FOR UPDATE SKIP LOCKED` row-locking clause, combined with an ordinary table, gives you an atomic “claim the next item and don’t let anyone else touch it” primitive — the same guarantee a dedicated queue product sells you, built out of two SQL keywords. This chapter builds a job queue from scratch: the schema, the atomic claim query, concurrent worker behaviour, stalled-job recovery, and a dead-letter path for jobs that keep failing.

This is not a toy exercise. This exact pattern — a status column, a claim query, a heartbeat, a dead-letter table — is what libraries like `river`, `oban` (Elixir), and countless in-house job runners implement on top of PostgreSQL in production.

The Scenario

Portsmith’s permitting office processes a steady stream of permit applications: building work, business licenses, public events, signage, and demolitions. Each application needs to move through a review pipeline, and the office wants that processing to happen

asynchronously and reliably — work should never be lost, never double-processed, and a crashed reviewer process shouldn't leave an application stuck in limbo forever.

The `jobs` table models this as a queue. Every row is one permit application awaiting review. A `status` column tracks its life cycle, a `priority` column lets safety-critical work (demolitions) jump ahead of routine work (sign permits), and a retry counter with a companion `dead_letter_jobs` table handles applications whose processing keeps failing.

Column	Purpose
`status`	`queued` → `in_progress` → `completed` (or back to `queued`, or dead-lettered)
`priority`	1 (most urgent — demolitions) to 5 (least urgent — sign permits)
`payload`	JSONB — the permit application details
`attempts` / `max_attempts`	Retry bookkeeping
`claimed_by` / `claimed_at`	Which worker has the job, and since when
`heartbeat_at`	Updated periodically by the worker while it holds the job

No extensions are required for this chapter — everything here is built on core PostgreSQL locking semantics.

Exercise Goals

By the end of this chapter you will be able to:

- Design a queue table schema, including a partial index tuned for the claim query's access pattern.
- Write the atomic `UPDATE ... FROM (SELECT ... FOR UPDATE SKIP LOCKED)` claim query, and explain why naive two-step approaches race.
- Observe, with two concurrent `psql` sessions, that `SKIP LOCKED` lets workers proceed past rows another worker is holding — and contrast that with the blocking behaviour of plain `FOR UPDATE`.
- Implement a heartbeat/timeout mechanism that reclaims jobs abandoned by a crashed worker.
- Route jobs that exhaust their retries into a dead-letter table.

- Benchmark claim throughput at different concurrency levels with `pgbench`.

Installation

This chapter needs nothing beyond what Chapter 1 already set up: PostgreSQL 16 and a Python 3.12 virtual environment with `psycopg`. If you skipped Chapter 1, see its Installation section. You will also use `pgbench` for Exercise 6, which ships with the standard PostgreSQL client tools (`postgresql-client-16` on Debian/Ubuntu).

Loading the Data

Run the seed script

From the `book/` directory, with the virtual environment active:

```
python data/ch03_seed.py
```

Expected output:

```
Connecting to: dbname=portsmith
Creating schema …
Inserting 45 jobs …
Done — 45 rows in jobs, all queued.
```

The seed script is self-contained — it does not depend on Chapter 1 or 2's data.

Verify the load

Open `psql portsmith` and run these checks.

Check 1 — table structure:

```
\d jobs
```

```
                                      Table "public.jobs"
    Column    |           Type           | Collation | Nullable
|             Default
--------------+--------------------------+-----------+----------
+---------------------------------
 id           | bigint                   |           | not null |
nextval('jobs_id_seq'::regclass)
```

```
 job_type     | text                     |           | not null |
 payload      | jsonb                    |           | not null |
 status       | text                     |           | not null |
'queued'::text
 priority     | smallint                 |           | not null |
5
 attempts     | integer                  |           | not null |
0
 max_attempts | integer                  |           | not null |
3
 created_at   | timestamp with time zone |           | not null |
clock_timestamp()
 claimed_at   | timestamp with time zone |           |          |
 claimed_by   | text                     |           |          |
 heartbeat_at | timestamp with time zone |           |          |
 completed_at | timestamp with time zone |           |          |
 last_error   | text                     |           |          |
Indexes:
    "jobs_pkey" PRIMARY KEY, btree (id)
    "idx_jobs_claim_order" btree (priority, created_at, id) WHERE
status = 'queued'::text
    "idx_jobs_status" btree (status)
Check constraints:
    "jobs_status_check" CHECK (status = ANY (ARRAY['queued'::text,
'in_progress'::text, 'completed'::text, 'failed'::text]))
```

Check 2 — job counts by type and priority:

```
        SELECT job_type, priority, COUNT(*) AS jobs
        FROM   jobs
        GROUP  BY job_type, priority
        ORDER  BY priority;
```

```
     job_type      | priority | jobs
-------------------+----------+------
 demolition_permit |        1 |    4
 business_license  |        2 |   12
 building_permit   |        3 |   15
 event_permit      |        4 |    8
 sign_permit       |        5 |    6
(5 rows)
```

Check 3 — everything starts queued:

```
        SELECT status, COUNT(*) FROM jobs GROUP BY status;
```

```
 status | count
--------+-------
 queued |    45
(1 row)
```

If all three match, proceed to the exercises.

> **Note:** If you re-run `ch03_seed.py` at any point during the exercises to reset to a clean state, it drops and recreates both `jobs` and `dead_letter_jobs`.

Exercises

Exercise 1 — Designing the Queue Schema

1.1 — Why a partial index

The claim query (which you'll write in Exercise 2) only ever looks at rows where `status = 'queued'`, ordered by `priority` then `created_at`. As the queue runs, the vast majority of rows will end up `completed` — a normal B-tree index on `(priority, created_at, id)` would faithfully index every one of those settled rows even though the claim query never looks at them.

`idx_jobs_claim_order` is a **partial index** — it only indexes rows matching `WHERE status = 'queued'`:

```
CREATE INDEX idx_jobs_claim_order
    ON jobs (priority, created_at, id)
    WHERE status = 'queued';
```

This keeps the index small regardless of how many historical jobs pile up in `completed` or `failed` state, because settled rows are never in it.

1.2 — Why `id` is part of the sort key, not just `created_at`

You might expect `ORDER BY priority, created_at` to be enough — oldest job in the highest-priority bucket goes first. But timestamps are not always unique. If two jobs are inserted in the same transaction, both can get an identical `created_at` (more on this below), and `ORDER BY` over tied values has no defined order. Appending the primary key, `id`, as a final tiebreaker guarantees a deterministic order even when timestamps collide:

```
ORDER BY priority ASC, created_at ASC, id ASC
```

> **A gotcha worth knowing:** `now()` returns the **transaction's** start time, not the current wall-clock time — every call to `now()` inside the same transaction returns the same value. The seed script originally used `created_at TIMESTAMPTZ DEFAULT now()`, and because all 45 rows were inserted in one transaction, every

> single row ended up with an *identical* `created_at`. The fix is `clock_timestamp()`, which returns the actual current time at the moment it's evaluated, differing row to row even within one transaction. `jobs.created_at` uses `clock_timestamp()` for exactly this reason. This is the same family of surprise as the JSONB `null` vs. SQL `NULL` gotcha from Chapter 1: a function name that looks interchangeable with another is not.

1.3 — Confirm the index is used

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT id
FROM   jobs
WHERE  status = 'queued'
ORDER  BY priority ASC, created_at ASC, id ASC
FOR UPDATE SKIP LOCKED
LIMIT  1;
```

```
 Limit  (cost=0.14..6.17 rows=1 width=24) (actual
time=0.022..0.022 rows=1 loops=1)
   Buffers: shared hit=3
   ->  LockRows  (cost=0.14..12.20 rows=2 width=24) (actual
time=0.021..0.021 rows=1 loops=1)
         Buffers: shared hit=3
         ->  Index Scan using idx_jobs_claim_order on jobs
(cost=0.14..12.18 rows=2 width=24) (actual time=0.010..0.010
rows=1 loops=1)
               Filter: (status = 'queued'::text)
               Buffers: shared hit=2
```

Now drop the index and run the identical query again inside a transaction you roll back (so the drop doesn't stick):

```
BEGIN;
DROP INDEX idx_jobs_claim_order;

EXPLAIN (ANALYZE, BUFFERS)
SELECT id
FROM   jobs
WHERE  status = 'queued'
ORDER  BY priority ASC, created_at ASC, id ASC
FOR UPDATE SKIP LOCKED
LIMIT  1;

ROLLBACK;
```

```
 Limit  (cost=9.51..9.53 rows=1 width=24) (actual
time=0.071..0.072 rows=1 loops=1)
   ->  LockRows  (cost=9.51..9.54 rows=2 width=24) (actual
time=0.070..0.071 rows=1 loops=1)
         ->  Sort  (cost=9.51..9.52 rows=2 width=24) (actual
```

```
time=0.067..0.068 rows=1 loops=1)
               Sort Key: priority, created_at, id
               Sort Method: quicksort  Memory: 27kB
               ->  Bitmap Heap Scan on jobs  (cost=4.16..9.50
rows=2 width=24) (actual time=0.018..0.027 rows=45 loops=1)
                     Recheck Cond: (status = 'queued'::text)
                     ->  Bitmap Index Scan on idx_jobs_status
(cost=0.00..4.16 rows=2 width=0) (actual time=0.013..0.013 rows=45
loops=1)
                           Index Cond: (status = 'queued'::text)
```

Without `idx_jobs_claim_order`, PostgreSQL still finds the queued rows (via `idx_jobs_status`), but it must then **sort all of them** to find the one with the lowest `(priority, created_at, id)` — an extra `Sort` node. With the tailored partial index, the rows are already stored in claim order, so PostgreSQL walks the index and stops at the first match. On a 45-row table the difference is invisible; on a busy production queue with a deep backlog, eliminating the sort on every single claim matters a great deal.

Exercise 2 — The Atomic Claim Query

2.1 — Why a naive two-step claim races

The tempting first approach is to `SELECT` a candidate row, then `UPDATE` it in a second statement:

```
        --   Do not do this — it has a race condition
        SELECT id FROM jobs WHERE status = 'queued' ORDER BY priority,
created_at, id LIMIT 1;
        -- ... application reads id = 7 ...
        UPDATE jobs SET status = 'in_progress' WHERE id = 7;
```

Between the `SELECT` and the `UPDATE`, nothing stops a second worker from running the exact same `SELECT`, reading the same `id = 7`, and also issuing the `UPDATE`. Both workers now believe they own job 7. This is a classic **check-then-act race condition** — the gap between reading and acting is exactly where two workers can interleave.

2.2 — `FOR UPDATE` closes the gap, but blocks

Locking the row as part of the `SELECT` closes the race:

```
        SELECT id FROM jobs WHERE status = 'queued' ORDER BY priority,
created_at, id
        FOR UPDATE LIMIT 1;
```

Now a second worker running the same query, in a separate transaction, **blocks** until the first worker's transaction commits or rolls back — correct, but it means every worker but one sits idle waiting for a lock instead of moving on to a different job. Section 3.1 demonstrates this.

2.3 — `SKIP LOCKED` lets workers move past each other

Adding `SKIP LOCKED` tells PostgreSQL: if the next candidate row is already locked by another transaction, don't wait for it — skip it and consider the row after it.

```
        SELECT id FROM jobs WHERE status = 'queued' ORDER BY priority,
created_at, id
        FOR UPDATE SKIP LOCKED LIMIT 1;
```

This is the piece that makes PostgreSQL usable as a concurrent queue: N workers can run this query at the same instant and each will walk away with a *different* row, with no blocking and no double-claims.

2.4 — The full atomic claim: lock, update, and return in one statement

Locking the row is only half the job — you still need to mark it `in_progress` before releasing the lock, and you want the whole thing to happen as a single round trip. Combine the `SELECT ... FOR UPDATE SKIP LOCKED` with the `UPDATE` using a CTE:

```
        WITH next_job AS (
            SELECT id
            FROM   jobs
            WHERE  status = 'queued'
            ORDER  BY priority ASC, created_at ASC, id ASC
            FOR UPDATE SKIP LOCKED
            LIMIT  1
        )
        UPDATE jobs
        SET    status       = 'in_progress',
               claimed_at   = now(),
               claimed_by   = 'demo-worker',
               heartbeat_at = now(),
               attempts     = attempts + 1
        FROM   next_job
        WHERE  jobs.id = next_job.id
        RETURNING jobs.id, jobs.job_type, jobs.payload ->>
'application_id' AS application_id,
                  jobs.priority, jobs.attempts, jobs.status;

 id |     job_type      | application_id | priority | attempts |
status
```

```
----+------------------+---------------+----------+----------
+-------------
  1 | demolition_permit | DP-2024-0001   |        1 |        1 |
in_progress
(1 row)
```

This single statement is the entire claim operation: find the best candidate, skip anything locked, mark it claimed, and hand back its data — atomically, with no window for a race. This is exactly the query `data/ch03_worker.py` runs (see `CLAIM_SQL`). Run it again and the demolition permit at `id = 2` comes back next — `id = 1` is now `in_progress`, so it's no longer a candidate.

> Roll this back if you ran it directly in `psql` and want to keep the queue clean for later exercises: wrap it in `BEGIN; ... ROLLBACK;`.

Exercise 3 — Simulating Concurrent Workers

3.1 — `SKIP LOCKED` vs. plain `FOR UPDATE`, in two `psql` sessions

Open two terminals with `psql portsmith` in each. In **Session A**, start a transaction, claim 5 rows, and hold the transaction open (don't commit yet):

```
        -- Session A
        BEGIN;
        SELECT id, job_type FROM jobs WHERE status = 'queued'
        ORDER BY priority, created_at, id
        FOR UPDATE SKIP LOCKED LIMIT 5;
```

```
 id |     job_type
----+------------------
  1 | demolition_permit
  2 | demolition_permit
  3 | demolition_permit
  4 | demolition_permit
  5 | business_license
(5 rows)
```

Leave that transaction open. In **Session B**, run the identical query:

```
        -- Session B (Session A is still open, holding locks on ids 1-5)
        BEGIN;
        SELECT id, job_type FROM jobs WHERE status = 'queued'
        ORDER BY priority, created_at, id
        FOR UPDATE SKIP LOCKED LIMIT 5;
```

```
 id |    job_type
----+-----------------
```

```
  6 | business_license
  7 | business_license
  8 | business_license
  9 | business_license
 10 | business_license
(5 rows)
```

Session B returns immediately with a **different** set of rows — it silently skipped 1–5 because they were locked, and moved on to the next five unlocked candidates. Commit or roll back both sessions to release the locks:

```
COMMIT;   -- run in both sessions
```

3.2 — Now try it with plain `FOR UPDATE` (no `SKIP LOCKED`)

Repeat the same two-session experiment, but drop `SKIP LOCKED`:

```
-- Session A
BEGIN;
SELECT id, job_type FROM jobs WHERE status = 'queued'
ORDER BY priority, created_at, id
FOR UPDATE LIMIT 5;
-- (leave this transaction open)

-- Session B — this will hang
BEGIN;
SELECT id, job_type FROM jobs WHERE status = 'queued'
ORDER BY priority, created_at, id
FOR UPDATE LIMIT 5;
```

Session B does not return. It is blocked, waiting for Session A's row locks to be released. Only once Session A runs `COMMIT` (or `ROLLBACK`) does Session B's query complete — and when it does, it returns the *same* five rows Session A had, now that they're unlocked again:

```
-- (Session B, after Session A commits)
 id |     job_type
----+------------------
  1 | demolition_permit
  2 | demolition_permit
  3 | demolition_permit
  4 | demolition_permit
  5 | business_license
(5 rows)
```

This is the difference in one sentence: **plain `FOR UPDATE` serializes workers through the lock; `SKIP LOCKED` lets them fan out across the table.** For a job queue, you always want the latter — a blocked worker is a wasted worker.

3.3 — Real concurrent workers with `ch03_worker.py`

Reset the data (`python data/ch03_seed.py`) if you ran the manual claim in Exercise 2.4 without rolling it back. Then launch two workers in the background, each capped to 10 jobs so they finish quickly:

```
        python data/ch03_worker.py --worker-id w1 --max-jobs 10 --fail-
rate 0 &
        python data/ch03_worker.py --worker-id w2 --max-jobs 10 --fail-
rate 0 &
        wait
```

Each prints a running log as it claims and completes jobs, e.g.:

```
[w1] claimed job 2 (demolition_permit, attempt 1/3): DP-2024-0002
— processing for 0.09s
[w1] job 2 completed
[w2] claimed job 1 (demolition_permit, attempt 1/3): DP-2024-0001
— processing for 0.12s
[w2] job 1 completed
```

Once both finish, confirm the split was clean — 20 jobs completed total, no job claimed by both workers:

```
        SELECT claimed_by, COUNT(*) FROM jobs WHERE status = 'completed'
        GROUP BY claimed_by ORDER BY claimed_by;
```

```
 claimed_by | count
------------+-------
 w1         |    10
 w2         |    10
(2 rows)
```

```
        SELECT status, COUNT(*) FROM jobs GROUP BY status;
```

```
  status   | count
-----------+-------
 completed |    20
 queued    |    25
(2 rows)
```

10 + 10 = 20, matching exactly the `--max-jobs 10` cap on each worker — no overlaps, no double-processing, no lost jobs.

Exercise 4 — Heartbeat and Stalled-Job Recovery

Claiming a job is only half the reliability story. What happens if the worker that claimed a job crashes, gets OOM-killed, or loses its network connection halfway through? The row is stuck at `status = 'in_progress'` forever unless something notices and puts it back.

4.1 — The heartbeat column

`heartbeat_at` exists for exactly this. A well-behaved worker updates it periodically while it holds a job (see the `while` loop in `process_job()` in `ch03_worker.py`, which sends a heartbeat roughly every 2 seconds during simulated processing). If a job has been `in_progress` for a long time *and* its heartbeat has gone stale, that's a strong signal the worker holding it is dead — a live worker would have updated it.

4.2 — Simulate a crashed worker

Pick any queued job and manually walk it through what a real claim would do, then simulate a crash by never sending a heartbeat again:

```
        UPDATE jobs
        SET    status = 'in_progress', claimed_at = now(),
               claimed_by = 'worker-crashed', heartbeat_at = now(),
               attempts = attempts + 1
        WHERE  id = 21;

        -- Simulate 5 minutes of silence from the "crashed" worker
        UPDATE jobs SET heartbeat_at = now() - interval '5 minutes' WHERE
id = 21;
```

4.3 — Run the reclaim sweep

```
        python data/ch03_reclaim.py --timeout 30
```

```
Connecting to: dbname=portsmith
  job 21 (building_permit) stalled — requeued (attempt 1)

Done: 1 requeued, 0 dead-lettered.
```

The sweep looks for `in_progress` rows whose `heartbeat_at` is older than `--timeout` seconds (`FIND_STALLED_SQL` in `ch03_reclaim.py`), and since this job's `attempts` (1) is still below `max_attempts` (3), it goes back to `queued`:

```
        SELECT id, status, attempts, claimed_by, heartbeat_at, last_error
        FROM   jobs WHERE id = 21;
```

```
 id | status | attempts | claimed_by | heartbeat_at
|                     last_error
```

```
----+--------+----------+------------+-------------
+-------------------------------------------------
 21 | queued |        1 |            |              | stalled: no
heartbeat since ... (last claimed by worker-crashed)
```

Notice `attempts` stayed at `1` — the reclaim did not reset it. The attempt the crashed worker used is still counted; the job doesn't get a free retry just because the worker that lost it never got to report failure honestly.

4.4 — Why the timeout, not an outright deadline

A fixed timeout on the *last heartbeat* (rather than on total processing time) lets jobs run arbitrarily long as long as they keep proving they're alive. A 10-minute report-generation job with a 30-second heartbeat interval will never be mistaken for stalled, while a worker that dies mid-task is detected within one missed heartbeat window. Run `ch03_reclaim.py` again immediately — it correctly finds nothing to do, since the reclaimed job now has a fresh `queued` state and no stale `in_progress` row exists:

```
No jobs stalled beyond 30s — nothing to do.
```

Exercise 5 — Dead-Lettering Exhausted Jobs

Some jobs will never succeed no matter how many times you retry them — a malformed application, a permanently invalid address, a bug that only triggers on one particular payload. Retrying forever wastes worker time and can mask a real problem. Once a job has used its last attempt, it belongs in `dead_letter_jobs`: out of the active queue, but preserved for a human to inspect.

5.1 — The `dead_letter_jobs` schema

```
        \d dead_letter_jobs

                    Table "public.dead_letter_jobs"
    Column    |           Type           | Collation | Nullable |
Default
--------------+--------------------------+-----------+----------
+---------
 id           | bigint                   |           | not null |
 job_type     | text                     |           | not null |
 payload      | jsonb                    |           | not null |
 priority     | smallint                 |           | not null |
 attempts     | integer                  |           | not null |
 max_attempts | integer                  |           | not null |
 created_at   | timestamp with time zone |           | not null |
 last_error   | text                     |           |          |
```

```
 failed_at    | timestamp with time zone |           | not null |
now()
```

It mirrors `jobs` but adds `failed_at` and drops the in-flight columns (`status`, `claimed_by`, `heartbeat_at`) that no longer mean anything once a job is out of the active queue.

5.2 — Push a job past its retry limit

Continuing with job 21 from Exercise 4 (now back to `queued` with `attempts = 1`), fast-forward it to its last attempt and let it stall again:

```
        UPDATE jobs
        SET    status = 'in_progress', claimed_at = now(),
               claimed_by = 'worker-crashed-2', heartbeat_at = now() -
interval '5 minutes',
               attempts = max_attempts
        WHERE  id = 21;

        python data/ch03_reclaim.py --timeout 30

Connecting to: dbname=portsmith
  job 21 (building_permit) exhausted retries — dead-lettered

Done: 0 requeued, 1 dead-lettered.
```

This time `attempts` (3) is no longer less than `max_attempts` (3), so the sweep's `DEAD_LETTER_SQL` runs instead: it `DELETE`s the row from `jobs` and `INSERT`s it into `dead_letter_jobs` in one CTE, so the row is never visible in neither-place or both-places, even under concurrent access.

```
        SELECT id FROM jobs WHERE id = 21;

 id
----
(0 rows)

        SELECT id, job_type, payload ->> 'application_id' AS
application_id,
               attempts, max_attempts, last_error, failed_at
        FROM   dead_letter_jobs WHERE id = 21;

 id |    job_type     | application_id | attempts | max_attempts
|                    last_error                     |
failed_at
----+-----------------+----------------+----------+--------------
+---------------------------------------------------
+-------------------------------
 21 | building_permit | BP-2024-0005   |        3 |            3 |
stalled: no heartbeat since ... (last claimed ...)  | 2026-07-12
22:15:45.613583-04
(1 row)
```

The exact same dead-lettering logic runs inline inside `ch03_worker.py` when a job fails organically (not via a stall) on its last attempt — see `DEAD_LETTER_SQL` in that script. Whether a job dies from an explicit failure or from going silent, it ends up in the same place, with a record of why.

Exercise 6 — Benchmarking Claim Throughput with `pgbench`

6.1 — A benchmark script that doesn't drain the queue

`pgbench` repeatedly runs a SQL script against the database for a fixed duration, from any number of concurrent client connections — exactly the tool for measuring how the claim query holds up under contention. The catch: if the script just claims a job and leaves it claimed, 45 rows disappear from the pool in well under a second and every worker after that finds nothing to do. `data/ch03_claim_bench.sql` works around this by claiming a job and then **immediately releasing it back to `queued`** in the same transaction:

```
        BEGIN;

        WITH next_job AS (
            SELECT id
            FROM   jobs
            WHERE  status = 'queued'
            ORDER  BY priority ASC, created_at ASC, id ASC
            FOR UPDATE SKIP LOCKED
            LIMIT  1
        )
        UPDATE jobs
        SET    status       = 'in_progress',
               claimed_at   = now(),
               claimed_by   = 'bench-' || :client_id,
               heartbeat_at = now()
        FROM   next_job
        WHERE  jobs.id = next_job.id
        RETURNING jobs.id AS claimed_id \gset

        UPDATE jobs
        SET    status = 'queued', claimed_at = NULL, claimed_by = NULL,
heartbeat_at = NULL
        WHERE  id = :claimed_id;

        COMMIT;
```

`:client_id` is a `pgbench` built-in variable — the number of the simulated client running this script. `\gset` captures the claimed row's `id` into a `pgbench` variable (`:claimed_id`) so the release step can target it. This keeps the pool at a constant 45 claimable rows for the whole benchmark run, so throughput reflects lock contention rather than the queue running dry.

6.2 — Run at increasing concurrency

```
pgbench -n -c 1  -j 1  -T 5 -f data/ch03_claim_bench.sql portsmith
pgbench -n -c 4  -j 4  -T 5 -f data/ch03_claim_bench.sql portsmith
pgbench -n -c 16 -j 16 -T 5 -f data/ch03_claim_bench.sql portsmith
```

`-c` is the number of simulated concurrent workers (clients), `-j` the number of `pgbench` threads driving them, and `-T 5` runs each test for 5 seconds. Results on the machine used to write this chapter:

Clients (`-c`)	tps	Avg. latency
1	799	1.25 ms
4	1,202	3.33 ms
16	2,570	6.23 ms

Your numbers will differ with your hardware — the shape of the result is what matters. Throughput climbs with concurrency because `SKIP LOCKED` lets additional workers keep finding unlocked rows to claim instead of queueing up behind each other; latency per transaction also climbs because more workers are contending for the same 45-row table and for CPU. Try dropping `idx_jobs_claim_order` (inside a `BEGIN; ... ROLLBACK;` block, as in Exercise 1.3) and rerunning the `-c 16` case — you should see tps fall, since every claim now pays for a `Sort` over all queued rows instead of an index walk.

6.3 — Where this stops being enough

`pgbench` here is exercising claim contention on 45 rows — a stand-in for "the working set the claim query actually scans." A real permitting-office queue might hold a very different shape of data (thousands of queued rows, heavy skew toward one `priority` bucket), and the honest way to benchmark your own workload is to seed a table that resembles it, not a 45-row toy. Chapter 20 (`pg_stat_statements` and Query Performance) picks this thread back up: rather than a synthetic benchmark, it measures and fixes real slow queries observed in production traffic.

Summary — What You Should Now Know

You built a working, concurrency-safe job queue entirely out of core PostgreSQL features. Here is a reference for the pieces:

Tool	What it does
`FOR UPDATE`	Locks selected rows; other transactions selecting the same rows **block** until released
`FOR UPDATE SKIP LOCKED`	Locks selected rows; other transactions **skip** already-locked rows instead of blocking
`WITH next_job AS (... FOR UPDATE SKIP LOCKED LIMIT 1) UPDATE ... FROM next_job`	The atomic claim pattern: find, lock, and mark a row in one round trip
Partial index `WHERE status = 'queued'`	Keeps the claim-path index small regardless of how many settled rows accumulate
`clock_timestamp()` vs. `now()`	`now()` freezes at transaction start; `clock_timestamp()` reflects real wall-clock time on every call
`heartbeat_at` + timeout sweep	Detects and recovers jobs abandoned by a crashed worker
`dead_letter_jobs`	Removes permanently-failing jobs from the active queue while preserving them for inspection
`pgbench -f script.sql`	Benchmarks a custom SQL workload at a chosen concurrency level

The key design insight from this chapter is that a reliable queue is not one clever query — it's the combination of an atomic claim, a way to detect workers that go silent, and a place for work that genuinely cannot succeed. PostgreSQL's row-locking primitives give you the first two almost for free; the dead-letter table is just a normal table.

The `jobs` table you built here is reused directly in later chapters: Chapter 13 adds a trigger that fires `NOTIFY` on status changes so listeners can react to queue activity in real time, Chapter 14 uses advisory locks alongside it for leader-election patterns, and Chapter 19 schedules the reclaim sweep you wrote by hand in Exercise 4 to run automatically with `pg_cron`.

Going further: the pattern in this chapter — status column, atomic claim, heartbeat, dead-letter — is exactly what PostgreSQL-backed job-queue libraries like `river` (Go) and `oban` (Elixir) implement, with more polish around scheduling, uniqueness constraints, and observability. If you outgrow a single-database queue — cross-database work distribution, guaranteed ordering at very high throughput, or consumer groups — that's the point at which a dedicated broker starts to earn its operational cost. For most applications below that scale, the table you just built is enough.